

Fișa de Lucru: Structurarea Codului. Ce sunt funcțiile (Subprogramele)?

Nivel: Clasa a IX-a

Timp de lucru estimat: 30 de minute

Ce vei învăța: Cum să împarți un cod lung în bucăți mai mici, ce este o funcție (subprogram) în Python, cum se transmit datele în interiorul ei, ce este o **conversie de tip** și cum utilizăm funcțiile predefinite precum `int()`.

1. De ce fragmentăm codul? (Breviar Teoretic)

Când ai de făcut o curățenie generală în casă, nu încerci să faci totul singur într-o secundă. Îți împarți sarcinile cu familia: un frate șterge praful, altul dă cu aspiratorul, iar cineva spală vasele. Fiecare se ocupă de o sarcină clară.

În programare, procedăm la fel. O problemă mare și complicată este împărțită în bucăți mai mici numite **Funcții** sau **Subprograme**.

O funcție poate fi vizualizată ca o mini-mașinărie dintr-o fabrică:

- Îi introducem materiile prime (**parametrii** sau datele de intrare).
- Mașinăria rulează un set de instrucțiuni stabilite de noi în interiorul ei (corpul funcției).
- Ne livrează produsul final (**rezultatul**), pe care îl trimite înapoi prin comanda **return**.

În Python, definim o funcție folosind cuvântul cheie **def** (de la *define* - a defini).

Un concept esențial: Ce este o conversie de tip (Type Conversion)?

În calculator, fiecare dată are un **tip** (o categorie). De exemplu:

- "10" este un **text**. Pe acesta nu îl putem aduna sau înmulți matematic!
- 10 este un **număr întreg** (sau `int`), bun pentru calcule.
- 10.5 este un **număr real/cu zecimale** (sau `float`).

Uneori, avem nevoie să transformăm o dată dintr-o categorie în alta. Această operațiune se numește **conversie de tip**.

Limbajul Python vine la pachet cu "mașinării" predefinite gata construite pentru conversii și alte calcule uzuale:

- **int(x)**: Este funcția principală de **conversie de tip în număr întreg**.
 - Dacă îi dăm un text format din cifre, îl transformă în număr: `int("12")` devine numărul 12.
 - Dacă îi dăm un număr cu zecimale, îi taie (truncchează) zecimalele, păstrând doar partea întreagă: `int(3.9)` devine 3.
- **abs(x)**: Returnează modulul (valoarea absolută) a unui număr. Transformă numerele negative în numere pozitive (ex: `abs(-7)` devine 7).
- **round(x)**: Rotunjește un număr real la cel mai apropiat număr întreg (ex: `round(3.6)` devine 4).

2. Să vedem codul! (Exemplu explicat pas cu pas)

Să construim o funcție numită **calculeaza_reducere** care ne ajută să aflăm prețul final (ca număr întreg, fără mărunțiș/zecimale) al unui produs din magazin după aplicarea unei reduceri.

```
# PASUL 1: DEFINIREA FUNCȚIEI noastre personalizate.
# Funcția are nevoie de doi parametri: pretul și procentul.
def calculeaza_reducere_intreaga(pret_initial, procent_reducere):
    # Calculăm câți bani economisim datorită reducerii
    economie = pret_initial * (procent_reducere / 100)
    pret_final_real = pret_initial - economie

    # Folosim funcția int() pentru o CONVERSIE DE TIP:
    # Transformăm prețul din număr cu zecimale (float) în număr întreg (int)
    pret_final_intreg = int(pret_final_real)

    # Returnăm (trimitem înapoi) prețul final convertit
    return pret_final_intreg

# --- AICI ÎNCEPE PROGRAMUL PRINCIPAL (codul care rulează efectiv) ---

pret_haina = 199.99 # Un preț cu zecimale (float)
reducere = 15      # Reducere de 15%

# APELĂM FUNCȚIA: îi trimitem valorile.
# Rezultatul returnat (care va fi un număr întreg) se salvează în "pret_de_plata".
pret_de_plata = calculeaza_reducere_intreaga(pret_haina, reducere)

print("Prețul rotunjit prin tăierea zecimalelor este:", pret_de_plata, "lei.")
# Pe ecran va apărea: Prețul rotunjit prin tăierea zecimalelor este: 169 lei.
```

3. Exerciții de verificare (Încearcă singur!)

Exercițiul 1: Dacă rulăm în Python instrucțiunea `print(int(3.9))` și apoi instrucțiunea `print(round(3.9))`, ce numere vor apărea pe ecran pentru fiecare dintre ele? Care este diferența de comportament din punct de vedere al conversiei și al rotunjirii matematice?

Exercițiul 2: Privește următorul program simplu:

```
def salut(num):
    mesaj = "Salut, " + num
    return mesaj

print(salut("Andrei"))
```

- **a)** Cum se numește elementul `num` aflat în paranteza definiției funcției (este parametru sau rezultat)?
- **b)** Ce text va apărea scris pe ecran după rularea acestui script?

Exercițiul 3 : Scrie o funcție proprie numită **converteste_si_aduna(text_a, text_b)** care:

1. Primește ca parametri două texte formate din cifre (de exemplu, "10" și "25").
2. Converteste ambele texte în numere întregi folosind funcția de conversie de tip `int()`.
3. Adună cele două numere obținute.
4. Returnează rezultatul adunării lor.
5. În programul principal, apelează această funcție cu textele "15" și "8" și afișează rezultatul pe ecran.

4. Barem de Evaluare și Corectare

- **Exercițiul 1:**
 - indicarea corectă a rezultatelor: 3 (pentru `int`) și 4 (pentru `round`).
 - explicarea diferenței: `int()` este o funcție de **conversie de tip** care elimină complet (trunchiază) zecimalele, pe când `round()` este o funcție matematică de rotunjire la cel mai apropiat întreg.
- **Exercițiul 2:**
 - răspunsul corect la cerința **a)**: nume este un **parametru** (sau argument de intrare).
 - răspunsul corect la cerința **b)**: pe ecran va scrie exact Salut, Andrei.
- **Exercițiul 3:**
 - definirea corectă a antetului funcției: `def converteste_si_aduna(text_a, text_b):`.
 - logica de conversie și adunare: realizarea conversiilor `int(text_a)` și `int(text_b)` urmată de returnarea sumei lor.
 - apelarea corectă în programul principal cu valori de tip text (cu ghilimele) și afișarea rezultatului: `print(converteste_si_aduna("15", "8"))`.